

Establishing quick EPICS support for new hardware using Python tools

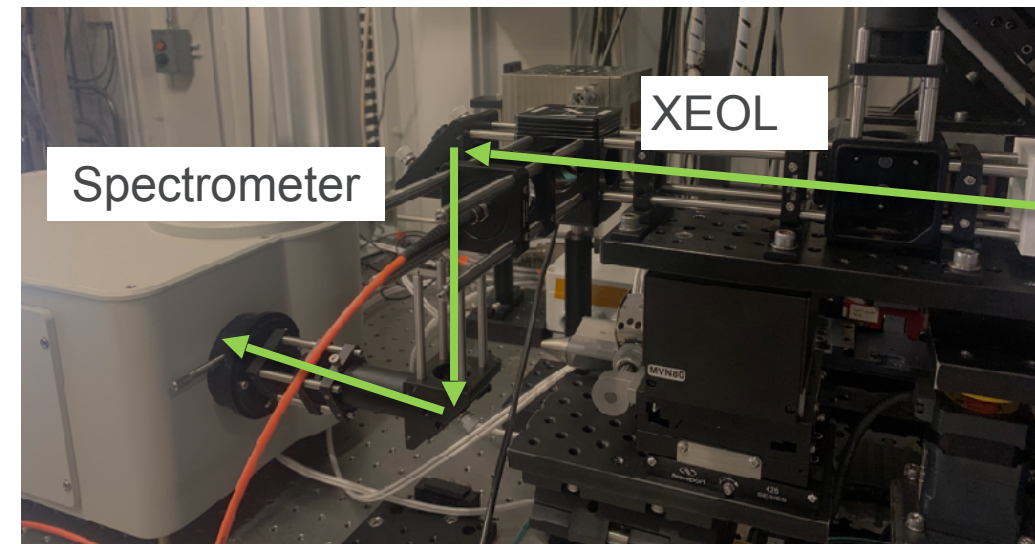
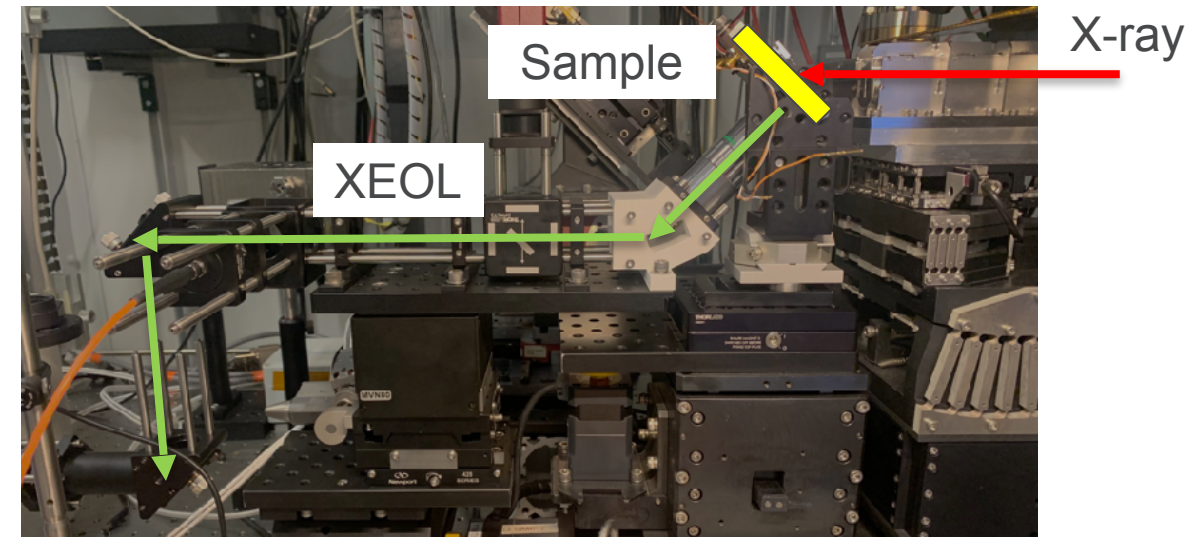
NAVEED RAHMAN – XSD-MIC
TWG Meeting

Showcasing how to setup EPICS support for unsupported hardware using Python-based tools

- Motivation behind establishing EPICS support
- Python-based tools used for non-C/C++/EPICS aficionados
- Example setup for a Horiba spectrometer

XEOL EXPERIMENTS AT BEAMLINE 34-ID-E

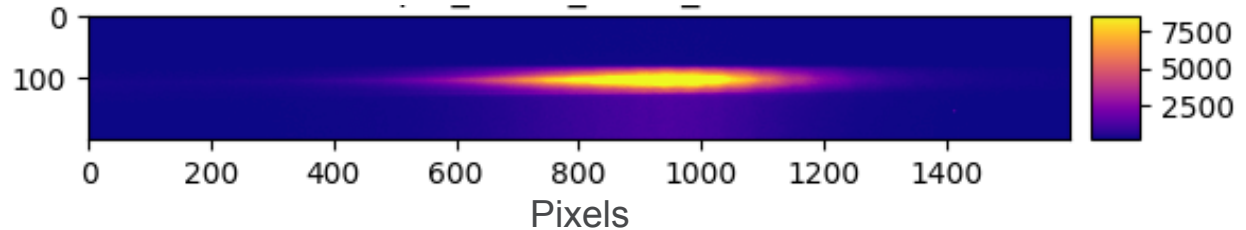
- XEOL (X-ray excited optical luminescence)
 - X-ray photon in/optical photon out
 - Collection optics and mirrors to redirect signal to downstream spectrometer
- Horiba iHR320 spectrometer with selectable diffraction gratings
- Measurement at each probed point results in a 2D image that is detected by an Andor Newton EMCCD coupled to the spectrometer



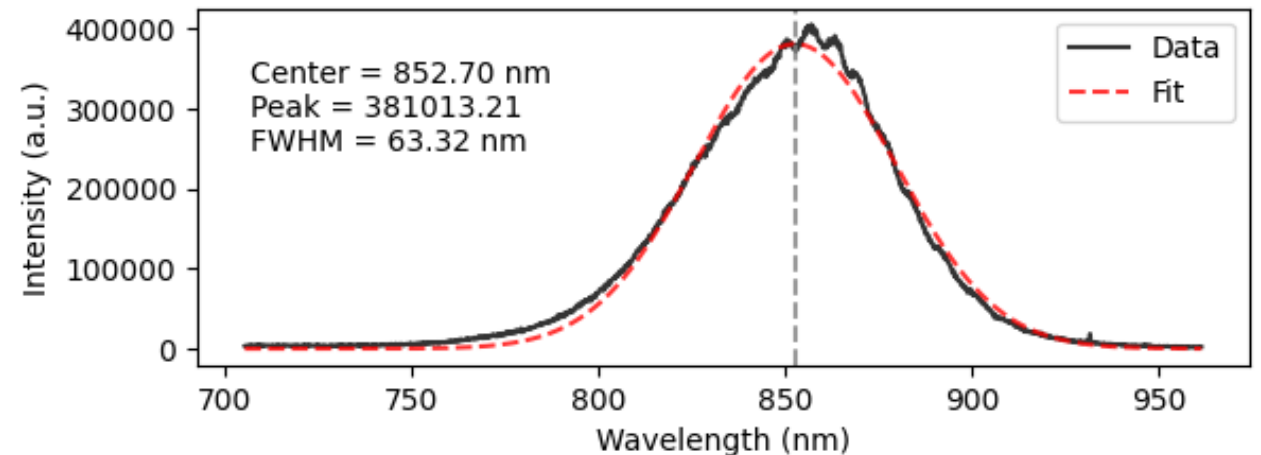
XEOL EXPERIMENTS AT SECTORS 2 & 34

- Measurement at each probed point results in a 2D image that is detected by an Andor Newton EMCCD coupled to the spectrometer
- Spectrometer and CCD are unaware of each other
 - Horiba: vendor-supplied software
 - Andor: EPICS support (areaDetector)
- Post-processing **after** data collection
 - X-axis is converted from pixels to wavelengths using calibration data
 - 2D images converted to 1D spectral lines by summing over the rows
- **Ideal end-goal:** pixel-to-wavelength live conversion done through EPICS without needing offline calibration data

Measured 2D Image



Post-Processed Spectrum



BUT WHY DO WE EVEN NEED EPICS?

- Typical XEOL experiments at 2-ID-D & 34-ID-E
 - Only one pair of entrance and exit ports used
 - Center wavelength set based on sample type
 - Diffraction grating typically never changed
 - No need for wavelength scanning (2D images!)
- “Set-and-forget” setup means EPICS support is **not a mission critical need**, the vendor software is adequate
- However, EPICS support *would still be nice to have*
 - Automated metadata logging
 - No need for a dedicated Windows PC
 - More sophisticated experimental setups
 - Stylistic consistency!



MAKING LIFE EASIER: USING ALREADY ESTABLISHED TOOLS

- Horiba hardware (up until the last beamtime cycle) has had no established EPICS support
- Windows-based SDK is available from the vendor
 - Unfortunately, I am not too comfortable in C/C++!
- Setting up hard support is not mission critical, so coming up with a soft solution is preferred
 - Pythonic workflow as opposed to pure C/C++/EPICS
- General steps:
 1. Device support using *yaq* (hardware ↔ Python)
 2. EPICS support using *softioc* (Python ↔ EPICS)
 3. User interface using *Qt Designer* (EPICS ↔ end user)



Instead of writing our own device-specific code, let's cut out the middle man and use already established tools!

THE YAQ ECOSYSTEM

- yaq: **Y**et **A**nother **a**c**Q**uision tool
 - Developed by Sunden, Thompson, *et al.* from the University of Wisconsin-Madison
 - <https://yaq.fyi/>
- Simple interface that **only handles hardware communication**
- Communication occurs over TCP/IP with data serialized using Apache Avro
 - Available in many different languages (Python, C/C++, C#, Java, Perl, Ruby, PHP)
- Framework is split up into daemons and clients
 - Daemons are small background processes that handle all low-level communication
 - Clients are the interpreters exposed to the user (e.g. a Python shell)

Review of Scientific Instruments	ARTICLE	scitation.org/journal/rsi
The yaq project: Standardized software enabling flexible instrumentation		
Cite as: Rev. Sci. Instrum. 94, 044707 (2023); doi: 10.1063/5.0135255 Submitted: 17 November 2022 • Accepted: 28 March 2023 • Published Online: 12 April 2023		
Kyle F. Sunden, , Daniel D. Kohler, , Kent A. Meyer, , Peter L. Cruz Parrilla, , John C. Wright, , and Blaise J. Thompson^{a)} 		
AFFILIATIONS University of Wisconsin-Madison, Madison, Wisconsin 53706, USA ^{a)}Author to whom correspondence should be addressed: blaise.thompson@wisc.edu		
ABSTRACT Modern instrumentation development often involves the incorporation of many dissimilar hardware peripherals into a single unified instrument. The increasing availability of modular hardware has brought greater instrument complexity to small research groups. This complexity stretches the capability of traditional, monolithic orchestration software. In many cases, a lack of software flexibility leads creative researchers to feel frustrated, unable to perform experiments they envision. Herein, we describe Yet Another acQuision (yaq), a software project defining a new standardized way of communicating with diverse hardware peripherals. yaq encourages a highly modular approach to experimental software development that is well suited to address the experimental flexibility needs of complex instruments. yaq is designed to overcome hardware communication barriers that challenge typical experimental software. A large number of hardware peripherals are already supported, with tooling available to expand support. The yaq standard enables collaboration among multiple research groups, increasing code quality while lowering development effort.		

The yaq (Yet Another acQuision) tool developed by Sunden et al. (DOI: 10.1063/5.0135255).

EXAMPLES OF YAQ-SUPPORTED HARDWARE

- Daemons standardize device communication
 - A “get-position” command retrieves a translation stage’s current XYZ position, a spectrometer’s wavelength setting, etc.
- Many daemons already created for a variety of vendors, some of which are yet to be fully supported through EPICS (Thorlabs)
- Luckily, Horiba is present on this list!

yaq/hardware

This page notes each model of hardware that can be driven via a yaq daemon. Many yaq daemons implement protocols that are capable of driving a wide variety of hardware, so the complete list of working hardware is in-theory much longer than seen here.

```
acton/2150i
acton/sp-2300i
adafruit/2348
andor/neo-5-5
becker-hickl/spc130
bronkhorst/el-flow
brooks/gf40
brooks/gf80
brooks/sla5800
dwyer/16b
dwyer/16c
ealing/37-1070-000
edac/epu452
ekspla/nt-342
gage/cse8442
horiba/ihr-320
horiba/microhr
idquantique/id100-50
kd-scientific/legato100
labjack/t4
labjack/t7-pro
lightcon/orpheus-hp
lightcon/orpheus-twins
lightcon/topas-800
```

<https://yaq.fyi/hardware/>



Brooks mass flow controller



RIGOL oscilloscope



Thorlabs power meter

PYTHON SOFTIOC MODULE

- *yaq* provides device support for establishing communication between hardware and client, next step is communication between a client and EPICS
- *softioc*: Python-based module to easily establish an EPICS IOC
 - <https://github.com/dls-controls/pythonSoftIOC>
- Typical workflow:
 1. Usual Python import commands
 2. Create EPICS records
 3. Initialize EPICS IOC and listen to device in the background

1

```
# Import the basic framework components.  
from softioc import softioc, builder  
import cothread
```

2

```
# Set the record prefix  
builder.SetDeviceName("MY-DEVICE-PREFIX")  
  
# Create some records  
ai = builder.aIn('AI', initial_value=5)  
ao = builder.aOut('AO', initial_value=12.45, on_update=lambda v: ai.set(v))
```

3

```
# Boilerplate get the IOC started  
builder.LoadDatabase()  
softioc.iocInit()  
  
# Start processes required to be run after iocInit  
def update():  
    while True:  
        ai.set(ai.get() + 1)  
        cothread.Sleep(1)  
  
cothread.Spawn(update)  
  
# Finally leave the IOC running with an interactive shell.  
softioc.interactive_ioc(globals())
```

HORIBA SPECTROMETER SETUP

- Using `yaq` + `softioc`, the general workflow for setting up EPICS support for the Horiba spectrometer:

- Create a configuration (.toml) file specifying TCP port and gratings
 - Run the `yaqd-horiba` daemon in the background
- Create Python script using the `softioc` module
 - Connect to device at specified port
 - Create EPICS records (e.g. analog input/output for center wavelength)
 - Create async while event for asynchronous channel access
- Get/set PVs through external Python/EPICS shell

Step 1

```
1 [mono]
2 port=39876
3 [mono.gratings]
4 [mono.gratings.mirror]
5 lines_per_mm=0
6 index=0
7 [mono.gratings.gr300]
8 lines_per_mm=300
9 index=1
10 [mono.gratings.gr600]
```

Step 2a

```
5 import asyncio
6 import numpy as np
7 import os
8 import time
9 import yaqc
10
11 from softioc import alarm, asyncio_dispatcher, builder, softioc
12
13
14 # Connect to Horiba daemon
15 c = yaqc.Client(39876)
```

Step 2b

```
50 # Wavelength position
51 v rbv_pos = builder.aIn(
52     'POSITION-RBV',
53     initial_value=c.get_position(),
54     EGU='nm',
55     PREC=2,
56 )
57 v rec_pos = builder.aOut(
58     'POSITION',
59     initial_value=init_position,
60     on_update=lambda v: c.set_position(v),
61     DRVL=0,
```

PV
name

Step 2c

```
107 # Status update
108 v async def update_status():
109     while True:
110         rbv_status.set(c.busy())
111         if c.busy():
112             rbv_status_label.set('Spectrometer busy...', s
113         else:
114             rbv_status_label.set('Spectrometer ready!', s
115
116         rbv_pos.set(c.get_position())
117         rbv_slit.set(c.get_front_entrance_slit())
118         rbv_mirror.set(c.get_exit_mirror())
119         rbv_turret.set(c.get_turret())
```

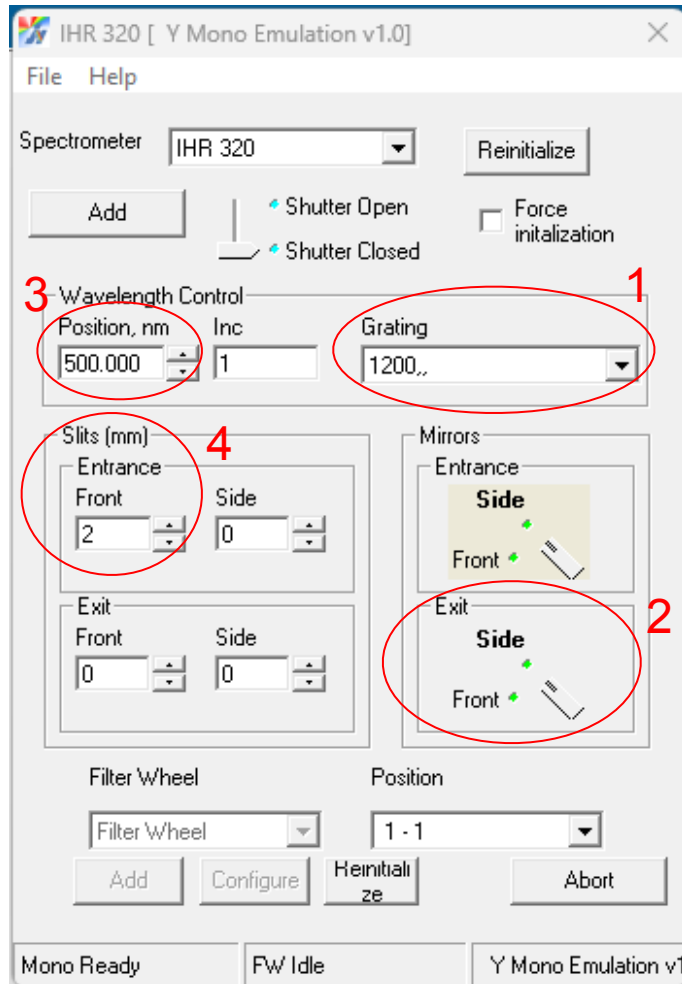
<https://github.com/narahma2/horiba-IOC/>

Step 3

```
>>> c.get_position()
0.0
>>> c.set_position(800)
>>> c.get_exit_mirror()
'front'
>>> c.set_exit_mirror('side')
>>> c.get_exit_mirror()
'side'
>>> c.set_exit_mirror('front')
>>> c.get_exit_mirror()
'front'
```

TRANSLATING THE VENDOR UI

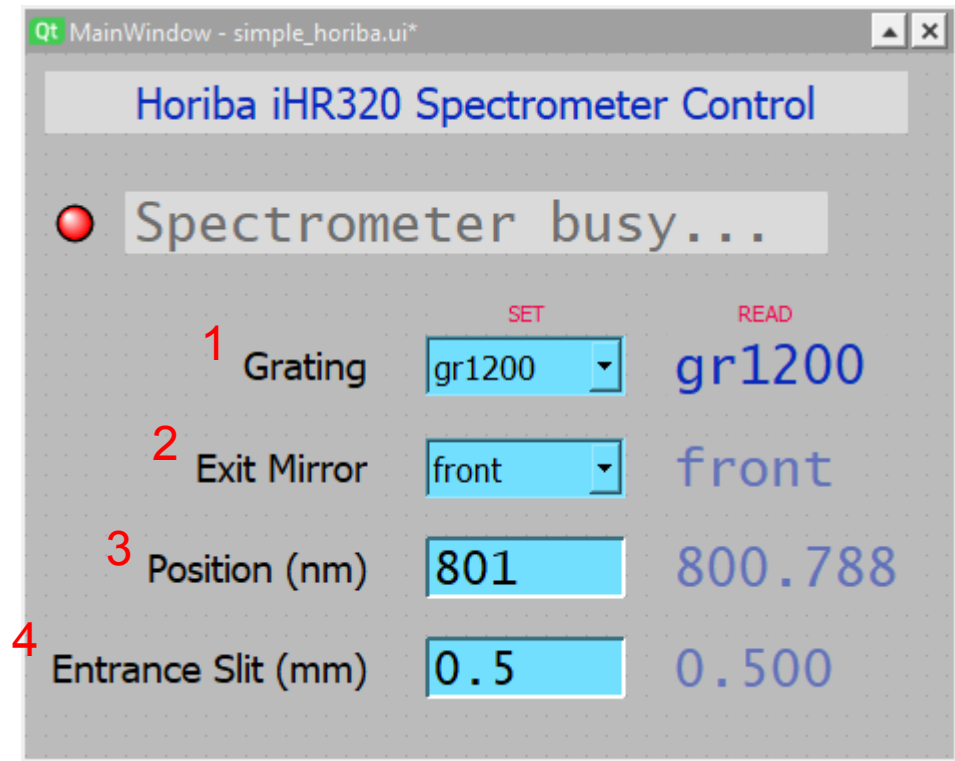
Vendor UI



yaq + softioc



Custom EPICS-Compatible UI



SUMMARY

- Showcased a pure Python workflow for establishing EPICS support using *yaq* + *softioc*
 - Example use-case given for a Horiba spectrometer, however the general workflow can be abstracted to any other device
- Possible next steps?
 - Incorporate calibration routine into UI
 - Combine areaDetector and new Horiba IOC for live spectral post-processing during measurements
 - Establish wavelength scanning record for 1D spectral measurements
- Thanks to...
 - Ross Harder for motivation to establish Horiba support
 - Dave Cyl + Fabricio Marin for previous help with understanding the EPICS ecosystem
 - The *yaq* team for doing most of the hard work in device setup!
 - All of you for your attention!